

```
# climate_env.def
Bootstrap: docker
From: python:3.11-slim

%post
    apt-get install -y cdo \
        libnetcdf-dev libhdf5-dev
    pip install xarray==2024.2.0 \
        dask==2024.2.0 netCDF4
    mkdir -p /leonardo /scratch

%environment
    export LC_ALL=C.UTF-8

%test
    python3 -c 'import xarray'
    cdo --version

%labels
    Version 1.0.0
    Description HPC preprocessing
```

Building Your First Image

Lecture 2 of 4

Definition files, Docker imports, NVIDIA NGC, sandbox, best practices

DATA SCIENCE AND ARTIFICIAL INTELLIGENCE – University of Trieste
Academic Year 2025/2026
Target platform: Leonardo @ CINECA

Two Paths to a Container Image

Strategy A is fast; Strategy B gives complete reproducible control

Strategy A - Pull from Registry

```
# Docker Hub
singularity pull \
  blast.sif docker://ncbi/blast:latest

# NVIDIA NGC
singularity pull \
  gromacs.sif docker://nvcr.io/\
  hpc/gromacs:2024.1

# Sylabs Cloud Library
singularity pull \
  library://ubuntu:22.04
```

- Fast: automatic OCI → SIF conversion
- Leverages 10M+ Docker Hub images

Strategy B Build from .def File

```
# On your laptop (requires sudo)
sudo apptainer build \
  my_env.sif my_env.def

# Transfer to Leonardo
scp my_env.sif \
  user@login.leonardo.cineca.it:\
  $SCRATCH/containers/

# Run on Leonardo
singularity exec \
  $SCRATCH/containers/my_env.sif cmd
```

- Complete control over every installed package
- Basis for reproducible, version-controlled research

The Definition File: Structure Overview

Header (what base) + Sections (what to do)

Section	When it runs	Purpose
%arguments	Build time (template)	Define reusable variables with {{ VAR }} syntax
%setup	Build on HOST first	Run commands on the host before container is created
%files	Build	Copy files from host into container filesystem
%post	Build INSIDE container as root	Install packages, compile software, configure THE KEY SECTION
%environment	Runtime (every startup)	Set environment variables visible at container launch
%runscript	singularity run	Default command when container is invoked with run
%test	After build	Smoke test to verify the image is correct
%labels	Metadata	Author, Version, Description inspectable with singularity inspect
%help	singularity run --help	Usage instructions for the container

Header Example

```
# Most common: Docker Hub
Bootstrap: docker
From: ubuntu:22.04

# With template variable
Bootstrap: docker
From: ubuntu:{{ VERSION }}

%arguments
VERSION=22.04
```

Bootstrap Sources

```
docker      → Docker Hub / registry
library     → Sylabs Cloud Library
localimage  → existing .sif file
scratch     → bare filesystem
oras        → OCI registry artifact
yum/apt     → distro package manager
debootstrap → Debian bootstrap
```

The %post Section: Where Real Work Happens

Runs inside the container as root during build: install everything here

```
Bootstrap: docker
From: ubuntu:22.04

%post
# System packages
apt-get update -y
apt-get install -y --no-install-recommends \
    build-essential gfortran \
    python3 python3-pip \
    libopenblas-dev wget ca-certificates
# CRITICAL: clean up to reduce SIF size
apt-get clean
rm -rf /var/lib/apt/lists/*

# Python packages with PINNED versions
pip3 install --no-cache-dir \
    numpy==1.26.4 scipy==1.13.0 \
    matplotlib==3.8.3

# Create Leonardo mount points
mkdir -p /leonardo /scratch /work
```

Critical %post practices

apt-get clean after install

Removes downloaded .deb files: reduces SIF size by 30–50%

rm -rf /var/lib/apt/lists/*

Clears package index cache: another 50-200 MB saved

--no-install-recommends

Skips optional dependencies - install only what's needed

Pin package versions

numpy==1.26.4 not numpy. Reproducibility depends on exact versions

mkdir -p /leonardo /scratch

Without these mount points, Apptainer cannot bind-mount Leonardo's Lustre

Test inside container

Run a quick sanity check at the end of %post; fail fast during build

Complete Definition File: Scientific Python

Climate analysis environment: Python 3.11 + CDO + xarray + dask

```
Bootstrap: docker
From: python:3.11-slim

%post
    apt-get update -y
    apt-get install -y --no-install-recommends \
        cdo libnetcdf-dev libhdf5-dev gcc g++
    apt-get clean
    rm -rf /var/lib/apt/lists/*
    pip install --no-cache-dir \
        numpy==1.26.4 scipy==1.13.0 \
        netCDF4==1.7.1 xarray==2024.2.0 \
        dask==2024.2.0 matplotlib==3.8.3
    mkdir -p /leonardo /scratch /work

%environment
    export LC_ALL=C.UTF-8
    export LANG=C.UTF-8

%test
    python3 -c 'import xarray; print(xarray.__version__)'
    cdo --version
```

```
%labels
    Author   MHPC_Student
    Version  1.0.0
    Description Climate preprocessing

%help
    Climate analysis container.
    Usage: singularity exec \
        climate_env.sif python3 script.py

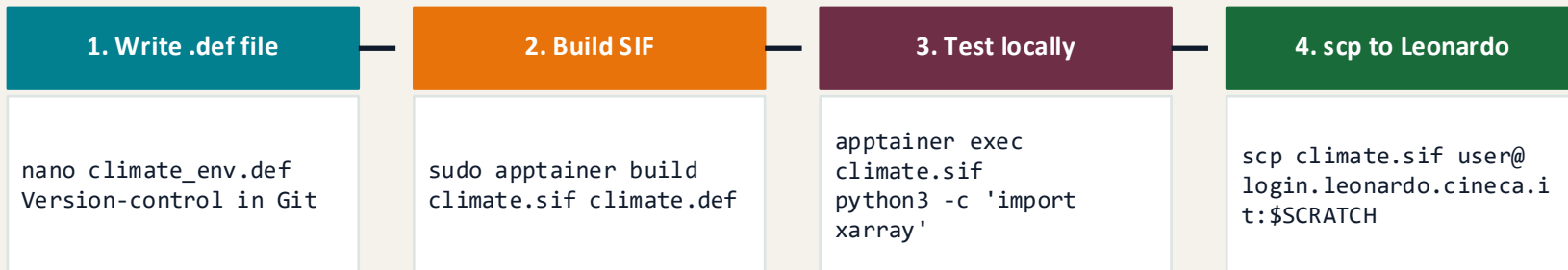
# — BUILD —————
# On your laptop:
sudo aptainer build \
    climate_env.sif climate_env.def

# — TEST —————
aptainer exec climate_env.sif \
    python3 -c 'import xarray'

# — TRANSFER —————
scp climate_env.sif \
    user@login.leonardo.cineca.it:\
    $SCRATCH/containers/
```

Build Workflow: Laptop → Leonardo

Build requires root; production runs on the cluster



On Leonardo production run:

```
singularity exec $SCRATCH/containers/climate_env.sif python3
$SCRATCH/my_analysis.py
```

Why build on laptop, not on Leonardo?

- sudo is required for singularity build and not available to users on Leonardo
- Internet access during %post (apt-get, pip) may be restricted on compute nodes
- Build is CPU-intensive: login nodes are shared; compute nodes are paid resources

Alternatives if you lack root access

- Apptainer 1.2+: rootless builds via --fakeroot on supported kernels
- Sylabs Remote Builder: build in cloud, download SIF
- Use existing Docker image + singularity pull (no build needed)

Converting Docker Images: Three Methods

From Docker Hub, from local tar archive, from your laptop's Docker daemon

```
# METHOD 1: Direct pull from Docker Hub (simplest)
singularity pull $SCRATCH/containers/blast.sif \
    docker://ncbi/blast:latest

# METHOD 2: Export from local Docker → tar → SIF
# (use when you have a custom image not on any public registry)
# Step 1 (laptop): export Docker image to tar archive
docker image save my_simulation:v2.1 -o my_sim_v2.1.tar
# Step 2 (laptop): convert tar to SIF
sudo apptainer build my_sim_v2.1.sif docker-archive://my_sim_v2.1.tar
# Step 3: transfer to Leonardo
scp my_sim_v2.1.sif user@login.leonardo.cineca.it:$SCRATCH/containers/

# METHOD 3: Pull from NVIDIA NGC (GPU-optimised, production-grade)
singularity pull $SCRATCH/containers/pytorch_24.01.sif \
    docker://nvcr.io/nvidia/pytorch:24.01-py3
singularity pull $SCRATCH/containers/gromacs_2024.sif \
    docker://nvcr.io/hpc/gromacs:2024.1
singularity pull $SCRATCH/containers/lammps.sif \
    docker://nvcr.io/hpc/lammps:patch_2Aug2023
```

NVIDIA NGC: Production-Grade HPC Containers

Maintained by NVIDIA; optimised for A100 on Leonardo's Booster partition: <https://catalog.ngc.nvidia.com/>

Container	NGC URI	Use Case
GROMACS 2024.1	<code>nvcr.io/hpc/gromacs:2024.1</code>	Molecular dynamics: CUDA + InfiniBand optimised
LAMMPS 2023	<code>nvcr.io/hpc/lammps:patch_2Aug2023</code>	Classical MD simulations
PyTorch 24.01	<code>nvcr.io/nvidia/pytorch:24.01-py3</code>	Deep learning + Python 3.10 + CUDA 12.3
TensorFlow 24.01	<code>nvcr.io/nvidia/tensorflow:24.01-tf2-py3</code>	TF2 with GPU support + Horovod MPI
NAMD 3	<code>nvcr.io/hpc/namd:3.0</code>	Biomolecular dynamics + GPU offloading
RAPIDS 24.02	<code>nvcr.io/nvidia/rapids:24.02-cuda12.0-runtime-ubuntu22.04</code>	GPU data science / cuDF / cuML
Modulus	<code>nvcr.io/nvidia/modulus/modulus:24.01</code>	Physics-ML / neural operators

Why NGC for GPU workloads: NVIDIA builds these images against specific CUDA versions and tests on A100 hardware. They include UCX/OpenMPI binaries compatible with InfiniBand. Starting from NGC saves weeks of manual compilation and dependency resolution.

Sandbox Mode: Interactive Development

Iterate on the image without a full rebuild cycle

```
# Convert existing SIF → writable sandbox directory
sudo aptainer build --sandbox \
    my_sandbox/ my_python.sif

# Open interactive shell with write access
sudo aptainer shell --writable my_sandbox/

# Inside: install additional packages
Apptainer> apt-get install -y htop vim h5utils
Apptainer> pip3 install h5py zarr
Apptainer> python3 -c 'import h5py; print(h5py.__version__)'
Apptainer> exit

# Verify the changes work
aptainer exec my_sandbox/ \
    python3 -c 'import h5py; print("OK")'

# Satisfied? Freeze to immutable SIF
sudo aptainer build my_python_v2.sif my_sandbox/
```

SIF vs Sandbox

Property	SIF	Sandbox
Format	1 large file	Directory tree
Writable	No	Yes (--writable)
Lustre perf.	Excellent	Poor (1000s files)
Production	Yes	No
Development	No	Yes

WARNING: Never place sandbox directories on Leonardo's \$SCRATCH or \$WORK. A sandbox for a moderately complex image can create 500,000+ small files — catastrophic for Lustre metadata servers. Build sandboxes on your laptop only.

Best Practices for Reproducible Research Images

These habits separate a research artefact from a throwaway script

Practice	Implementation	Why it matters
Version everything	From: ubuntu:22.04 not :latest	ubuntu:latest changes monthly: breaks reproducibility
Pin package versions	pip install numpy==1.26.4	'latest' numpy may introduce breaking changes
Document in %labels	Author, Version, Description	Inspectable with singularity inspect --labels
Test in %test section	Verify critical imports at build time	Catch broken builds before transferring to cluster
Minimise image size	apt-get clean + rm -rf /var/lib/apt/lists/*	Smaller SIF = faster transfer, less Lustre I/O
Store .def in Git	git add climate_env.def + git commit	The .def is the source of truth; .sif is the binary
Tag filenames with versions	gromacs-2024.1_cuda-12.2.sif	Avoids overwriting known-good images accidentally
Create mount points	mkdir -p /leonardo /scratch /work in %post	Required for Apptainer bind mounts on Leonardo

Environment Variable Inheritance

Host → Container: automatic. Container-only: SINGULARITYENV_ / APPTAINERENV_ prefix

```
# Variables set on HOST are visible INSIDE container
export MY_DATA_DIR=/leonardo/scratch/.../data
export OMP_NUM_THREADS=28
export SLURM_CPUS_PER_TASK=28 # set by SLURM

# All of these are visible inside:
singularity exec my.sif printenv OMP_NUM_THREADS
# Output: 28
singularity exec my.sif printenv SLURM_CPUS_PER_TASK
# Output: 28

# Set variable ONLY inside the container:
export SINGULARITYENV_OMP_NUM_THREADS=56
singularity exec my.sif printenv OMP_NUM_THREADS
# Inside: 56 | Host: still 28
```

```
# Block a variable from entering
# the container:
singularity exec \
    --env-file=custom.env \
    my.sif cmd

# Prefix equivalence:
# SINGULARITYENV_X=val
# = APPTAINERENV_X=val
# Both work on Leonardo

# %environment in .def sets
# variables at EVERY startup
# (not just during build)
```

Key for HPC: OMP_NUM_THREADS, OMP_PROC_BIND, OMP_PLACES, KMP_AFFINITY, CUDA_VISIBLE_DEVICES, and all SLURM_* variables set by the job scheduler are automatically available inside the container. You do not need to configure anything special.

SINGULARITYENV_ prefix: use when you want a variable visible inside the container only, not on the host.

Inspecting Images and Managing Cache

Know what's in your image; keep cache off \$HOME

```
# Inspect labels and metadata
singularity inspect --labels my_env.sif
# Outputs:
# org.label-schema.version: 1.0.0
# Author: MHPC_Student

# Show embedded definition file
singularity inspect --deffile my_env.sif

# Show runscript
singularity inspect --runscript my_env.sif

# Shell into container for manual inspection
singularity shell my_env.sif
Apptainer> cat /etc/os-release
Apptainer> python3 --version
```

```
# Cache management
# Default: ~/.singularity/cache
# DANGER on Leonardo: fills $HOME

# ALWAYS redirect on Leonardo:
export SINGULARITY_CACHEDIR=\
    $SCRATCH/.singularity_cache
mkdir -p $SINGULARITY_CACHEDIR

# List cached items
singularity cache list

# Clean the cache
singularity cache clean
# or clean specific type
singularity cache clean --type=blob
```

\$HOME quota on Leonardo is 50 GB. A single GROMACS NGC container expands to ~8 GB of layer cache. Three or four containers will fill your home. Always set SINGULARITY_CACHEDIR to \$SCRATCH before the first singularity pull. Add it to your ~/.bashrc or your job script preamble.

Exercise 2: Building and Importing Images

Scientific context: climate modelling preprocessing with CDO + xarray + dask

Part 1: Build climate_env.sif [15 pts]

- Write climate_env.def with Python 3.11 + CDO + xarray + netCDF4
- `sudo aptainer build climate_env.sif climate_env.def`
- Verify %test passes: `xarray import + cdo --version`
- Measure final SIF size; inspect `--labels`

Part 2: Run on Leonardo [8 pts]

- scp to `$SCRATCH/containers` on Leonardo
- Write test_xarray.py: synthetic 2D temperature field
- Run via `salloc + singularity exec` on DCGP
- Verify synthetic_climate.nc appears on host `$SCRATCH`

Part 3: Docker Image Import [7 pts]

- `docker save nvcr.io/hpc/gromacs:2024.1 -o gromacs.tar`
- `sudo aptainer build gromacs.sif docker-archive://gromacs.tar`
- Compare .tar vs .sif sizes — explain SquashFS compression
- `singularity inspect --labels`: what CUDA version?

Exercise 2: Key Questions to Answer

Think through these before the solutions session

- Q1** Why do we create `/leonardo`, `/scratch`, `/work` in `%post` even though the container image never contains data from these paths?
- Q2** The Python script writes `synthetic_climate.nc` to `$SCRATCH`. You exit the container. The file is still there. Why? (Contrast: what if the script wrote to `/tmp` inside the container?)
- Q3** `gromacs_2024.1.tar` is ~14 GB; `gromacs_2024.1.sif` is ~9 GB. Where did 5 GB go? What algorithm is responsible?
- Q4** `docker://ubuntu:latest` and `docker://ubuntu:22.04` both exist on Docker Hub. Why should a research container definition NEVER use `:latest`?
- Q5** Why can't singularity build run on Leonardo's login nodes or compute nodes? What would happen if it could?

When Each Section Executes

Timeline: build time vs runtime critical for understanding %environment

BUILD TIME (on your laptop, with sudo)

%setup Runs on HOST filesystem (not inside container). Rarely needed.

%files Copies files from HOST into the container image.

%post THE main section: runs inside container as root. Install packages, compile, configure.

%test Runs inside container after %post: verify the build is correct. Fails build if non-zero exit.

RUNTIME (singularity exec / run)

%environment Variables sourced at every container startup. Available to all commands.

%runscript Executed only by 'singularity run'. Ignored by 'singularity exec'.

%startscript Executed by 'singularity instance start'. For persistent daemons.

Common confusion: %environment does NOT run during build, it is sourced by the shell at every container startup. Variables you set in %post are only visible during %post itself. To make a variable permanent in the container, set it in %environment (runtime) not %post (build time).

Reducing Image Size: Multi-Stage and Clean-Up

Large SIF files slow transfers and waste Lustre quota

```
# BEFORE: naive build - 3.2 GB SIF
Bootstrap: docker
From: ubuntu:22.04
%post
    apt-get install -y build-essential \
        cmake python3 python3-pip wget
    # Compile from source
    wget https://.../gromacs-2024.1.tar.gz
    tar xzf gromacs-2024.1.tar.gz
    cmake .. && make -j8 && make install
    # Forget to clean up!

# AFTER: clean build - 1.1 GB SIF
%post
    # ... (same install) ...
    # Clean up build artefacts:
    rm -rf /tmp/gromacs-* /tmp/*.tar.gz
    apt-get remove -y build-essential cmake
    apt-get autoremove -y
    apt-get clean
    rm -rf /var/lib/apt/lists/*
```

Size reduction checklist

apt-get clean after every install block

rm -rf /var/lib/apt/lists/* after apt-get

pip install --no-cache-dir always

Remove source tarballs after compile

Use python:3.11-slim not python:3.11

Use ubuntu:22.04 not ubuntu:22.04 full

Remove build tools (gcc, cmake) after use if runtime does not need them

Advanced: Spack Inside Containers

Declarative HPC software stack management inside Apptainer

Spack is a package manager for HPC that handles complex dependency graphs, compiler flags, and MPI variants automatically. Using Spack inside a container gives you: reproducible, documented software stacks that can be rebuilt exactly on any system.

```
# def file snippet: Spack inside container
Bootstrap: docker
From: ubuntu:22.04

%files
    spack.yaml    /spacking/spack.yaml

%post
    apt-get install -y build-essential \
        ca-certificates git curl python3
    git clone https://github.com/spack/spack.git
    source /spack/share/spack/setup-env.sh
    spack env activate -d /spacking/
    spack concretize
    spack install
```

```
# spack.yaml example
spack:
  specs:
    - openmpi@4.1.6+mpi \
      fabrics=ucx \
      schedulers=slurm
    - fftw@3.3.10 +mpi \
      +openmp %gcc@11
    - hdf5@1.14.3 +mpi
  config:
    install_tree:
      root: /opt/spack
```

Key Takeaways

Five points to carry into Next Lecture

1

%post is where real work happens

It runs inside the container as root during build. Always follow apt-get install with apt-get clean && rm -rf /var/lib/apt/lists/*.

2

singularity pull converts Docker → SIF automatically

Use docker://, docker-archive://, or nvcr.io/ URIs. The resulting SIF is immutable and Lustre-friendly.

3

Build on laptop, run on cluster

sudo is required for build. Build locally, scp the SIF. Never build inside SLURM jobs.

4

%environment ≠ %post for variables

%environment sources at every container startup. Variables set in %post disappear after build. Use %environment for permanent container variables.

5

The .def file is the source of truth

Version-control it in Git. Pin all versions. The .sif is the binary artefact — tag it with software versions in the filename.

What's Next

L3: OpenMP: Affinity and NUMA

OMP_PROC_BIND, OMP_PLACES, SLURM --cpu-bind, numactl.
DGEMM scaling study on DCGP nodes (2×56 core Intel Sapphire Rapids). Thread placement diagrams.

L4: MPI and GPU in Containers

Host MPI + singularity exec pattern. --nv GPU passthrough and CUDA driver injection. Hybrid MPI+OpenMP+GPU on A100 Booster. SAXPY bandwidth benchmark.

Exam: Container Cluster

Build container cluster with SLURM. Deploy and run benchmarks inside Apptainer.

Now: Exercise 2

Build climate_env.sif on your laptop. Transfer to Leonardo. Run xarray analysis. Import GROMACS from Docker archive. 30 points.